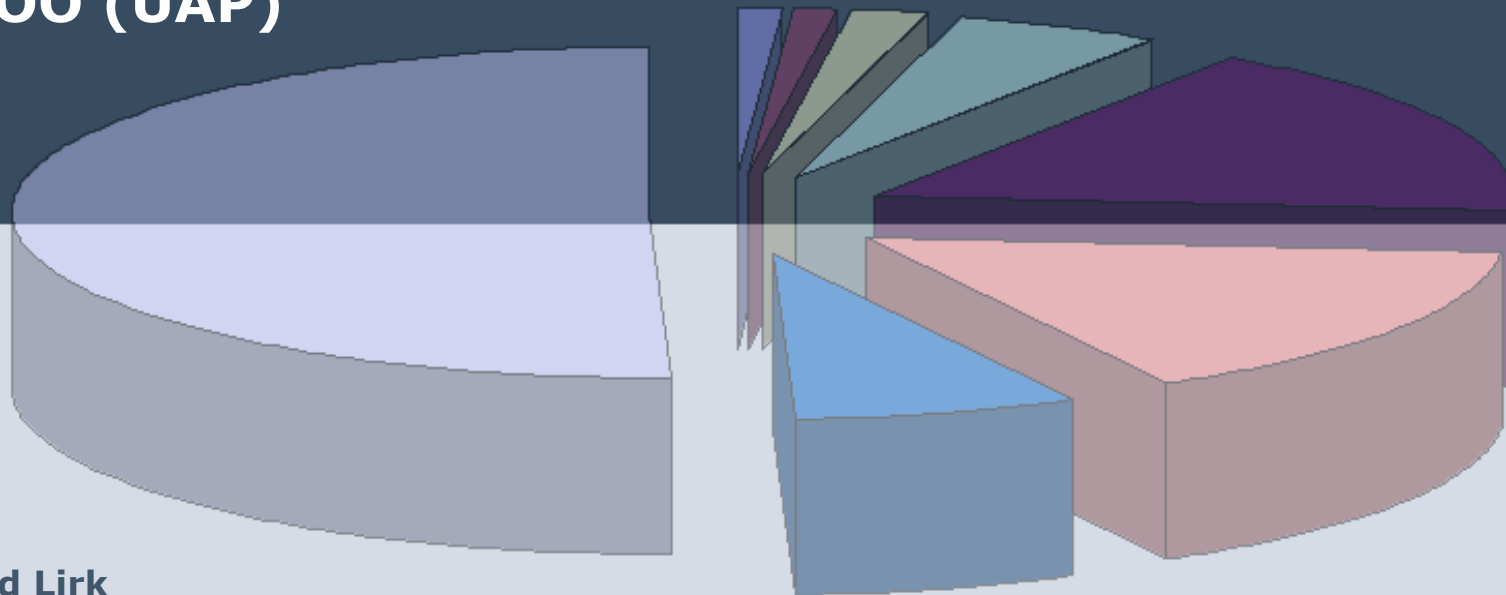


Statistik mit R und SPSS

R - Einführung

FH OÖ (UAP)



Gerald Lirk
E-Mail: gerald.lirk@fh-hagenberg.at



Inhalt

- ▶ Einleitung
- ▶ Datentypen
- ▶ Datenhandling
- ▶ statistische Tests
- ▶ Programmierung
- ▶ tidyverse: plyr, dplyr
- ▶ Grafiken



R vs Python

www.theknowledgeacademy.com

theknowledgeacademy

Python Programming



VS

R Programming



Python provides a more general approach to data science.

Python users are mostly Programmers and Developers.

Python is linear and smooth to learn.

Python provides flexibility to construct new models from scratch.

Python consists packages and libraries like pandas, scipy, scikit-learn, TensorFlow, caret.

R is mainly used for statistical analysis.

R users mainly consists of Scholars and R&D professionals.

R is difficult to learn at the beginning.

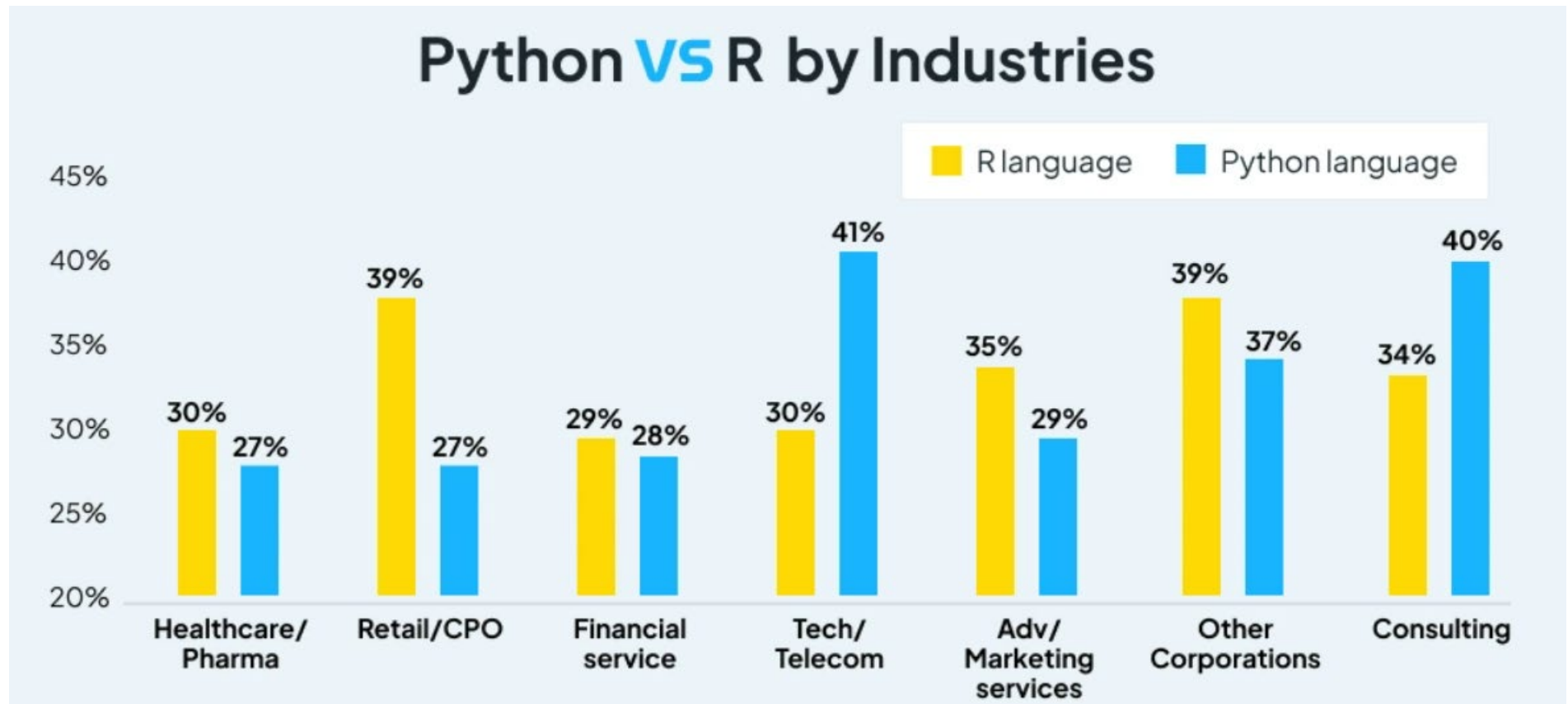
R provides flexibility to use available libraries.

R consists various packages and libraries like ttidyverse, ggplot2, caret, zoo.



R vs Python

geeksforgeeks (2025)



<https://www.geeksforgeeks.org/python/r-vs-python/>



R vs Python

DataCamp: Data Science Wars (2015)

 VS.  python	
History	
Creators	Creator
Ross Ihaka and Robert Gentleman	Guido Van Rossum
Release Year	Release Year
1995	1991
Must Knows	Must Knows
1. R is an implementation of S programming language (Bell Labs). 2. R's design and evolution is handled by the R-core group and R foundation. 3. R's software environment was written primarily in C, Fortran and R.	1. Python was inspired by C, Modula-3, and particularly ABC. 2. Python gets its name from the "Monty Python's Flying Circus" comedy series. 3. Python Software Foundation (PSF) takes care of Python's advances.



R vs Python

DataCamp: Data Science Wars (2015)

	VS.	 python
Purpose		
R focuses on better, user friendly data analysis, statistics and graphical models.		Python emphasizes productivity and code readability.
Used By?		
R has been used primarily in academics and research. However, R is rapidly expanding into the enterprise market.		Python is used by programmers that want to delve into data analysis or apply statistical techniques, and by developers that turn to data science.
<i>"The closer you are to statistics, research and data science, the more you might prefer R."</i>		<i>"The closer you are to working in an engineering environment, the more you might prefer Python."</i>



R vs Python

DataCamp: Data Science Wars (2015)

 VS.  python	
Usability	
Statistical models can be written with only a few lines.	Coding and debugging is easier to do in Python, mainly because of the "nice" syntax.
There are R stylesheets but not everyone uses them.	The indentation of the code affects its meaning.
The same piece of functionality can be written in several ways in R.	Any piece of functionality is always written the same way in Python.
Flexibility	
It is easy to use complex formulas in R. All kinds of statistical tests and models are readily available and easily used.	Python is flexible for doing something novel that has never been done before. Developers can also use it for scripting a website or other applications.



Graphical Capabilities



IPython Notebook

A picture says more than a thousand words

Visualized data can be understood more efficiently and effectively than the raw numbers alone.

R + visualization
= perfect match



ggplot2 To make pretty graphs including the opportunity to use grammar of graphics to create layered, customizable plots

lattice To easily display multivariate relationships

rCharts To create, customize and publish interactive javascript visualizations from R

googleVis To use Google Chart tools to visualize data in R

ggvis To implement interactive grammar of graphics, while rendering in a web browser

e.g.: Visualizing Facebook friends with R



Bundle your analysis in one file

The IPython Notebook makes it easier to work with Python and data.

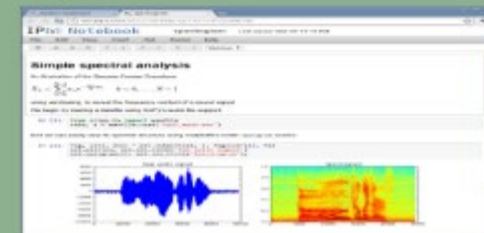
Simplify your workflow when working with data in Python

It's a combination of:

Interactive python exploration,
prewritten programs, text, and equations for
documentation in one environment

Share notebooks with colleagues
without having them install anything.

The IPython notebook drastically reduces the overhead of organizing code, output, and notes files, which allows to spend more time doing real work.



R, Lingua Franca of Statistics



Python, A Multi-Purpose Language

Developed by statisticians, for statisticians

Statisticians communicate ideas and methods for statistical analysis through R code and packages.

Statisticians, engineers and scientists without computer programming skills find it easy to use.

Increasing industry adoption...

R is used in finance, pharmaceuticals, media and marketing; In this last area, R's on the rise as a business analytics tool.

"The number one value to businesses in using R is access to talent"

Google



... And widespread use in academia

R is experiencing a rapid growth, solidifying its position in third place as software used in scholarly articles, right after SAS and SAP.

Ready To Work!

As a common, easy-to-understand language that is known by many programmers, Python also brings people with different backgrounds together.

For example,

Some organizations that didn't want to hire or had difficulties to hire new data scientists (re)trained their existing employees to use Python instead.

This means that Python is a production ready language: it has the capacity to be a single tool that integrates with every part of your workflow!





- ▶ Download unter rstudio.org
- ▶ Kostenlose IDE
 - ▶ R-Konsole
 - ▶ Editorfenster für R-Skripts
 - ▶ Plots, Hilfeseiten, Historie. Workspace-Übersicht, etc.
- ▶ RStudio als Editor
 - ▶ Befehl ausführen: `Strg + Enter` (oder `Strg + R`)
 - ▶ Ähnliche Befehle: `Strg + Leertaste`
 - ▶ Hilfeseiten: `F1`
- ▶ `Tools -> Global Options .... -> Pane Layout`



Hilfe

help()

- ▶ Alternativ zu `help` kann man auch `?` Verwenden
 - `> help(mean) #bzw.`
 - `> ?mean`
- ▶ Natürlich gibt es auch Hilfe zur Hilfe
 - `> help(help) #bzw.`
 - `> ?help`
- ▶ Sehr beliebt ist auch die Hilfeseite zu `par()`. Sie beschreibt eine Vielzahl von Parameter, die an Grafikfunktionen (z.B. `plot()`), weitergegeben werden können.
 - `> help(par) #bzw.`
 - `> ?par`



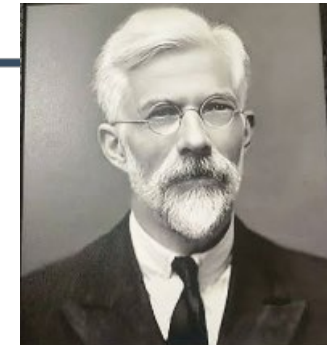
Hilfe

Links

- ▶ Offizielle Homepage R-Projekt: www.r-projekt.org
- ▶ CRAN-Netzwerk mit aktuellen Versionen: cran.r-projekt.org
- ▶ Suchmaschine von S. Goodman: www.rseek.org
- ▶ R bloggers: www.r-bloggers.com
- ▶ Stack Overflow mit R-Filter: stackoverflow.com/questions/tagged/r
- ▶ E-Mail-Listen
 - ▶ R-help@stat.math.ethz.ch
 - ▶ R-devel@stat.math.ethz.ch

Datensätze

Iris



International Father of Statistics
Sir Ronald Aylmer Fisher's
Born 17, February 1890
Died 29, July 1962

▶ 1936 von R. Fisher in
*„The use of multiple measurements
in taxonomic problems“*

▶ $n = 150$



Iris setosa



Iris virginica



Iris versicolor



Datensätze

Iris

► Laden des Iris-Datensatzes

```
library(datasets)
```

```
data(iris)
```

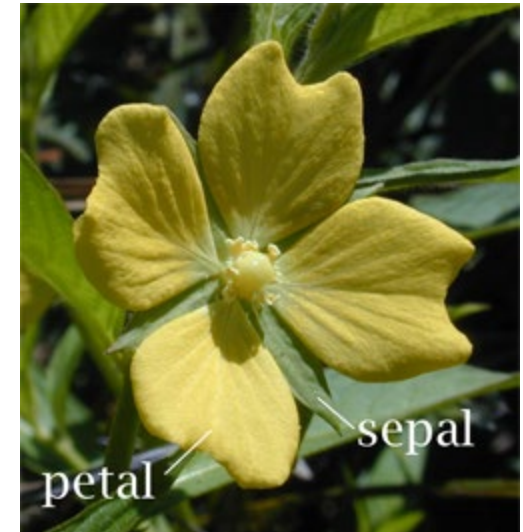
► Inhalt

```
names(iris)
```

```
[1] „Sepal.Length“ „Sepal.Width“ „Petal.Length“  
„Petal.Width“ „Species“
```

► 4 metrisch-stetig

► 1 nominal





Datensätze

Iris

► Ansicht des Iris-Datensatzes

```
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
summary(iris)
```

Sepal.Length		Sepal.Width		Petal.Length		Petal.Width		Species	
Min.	:4.300	Min.	:2.000	Min.	:1.000	Min.	:0.100	setosa	:50
1st Qu.	:5.100	1st Qu.	:2.800	1st Qu.	:1.600	1st Qu.	:0.300	versicolor	:50
Median	:5.800	Median	:3.000	Median	:4.350	Median	:1.300	virginica	:50
Mean	:5.843	Mean	:3.057	Mean	:3.758	Mean	:1.199		
3rd Qu.	:6.400	3rd Qu.	:3.300	3rd Qu.	:5.100	3rd Qu.	:1.800		
Max.	:7.900	Max.	:4.400	Max.	:6.900	Max.	:2.500		

R
AUFGABE 1

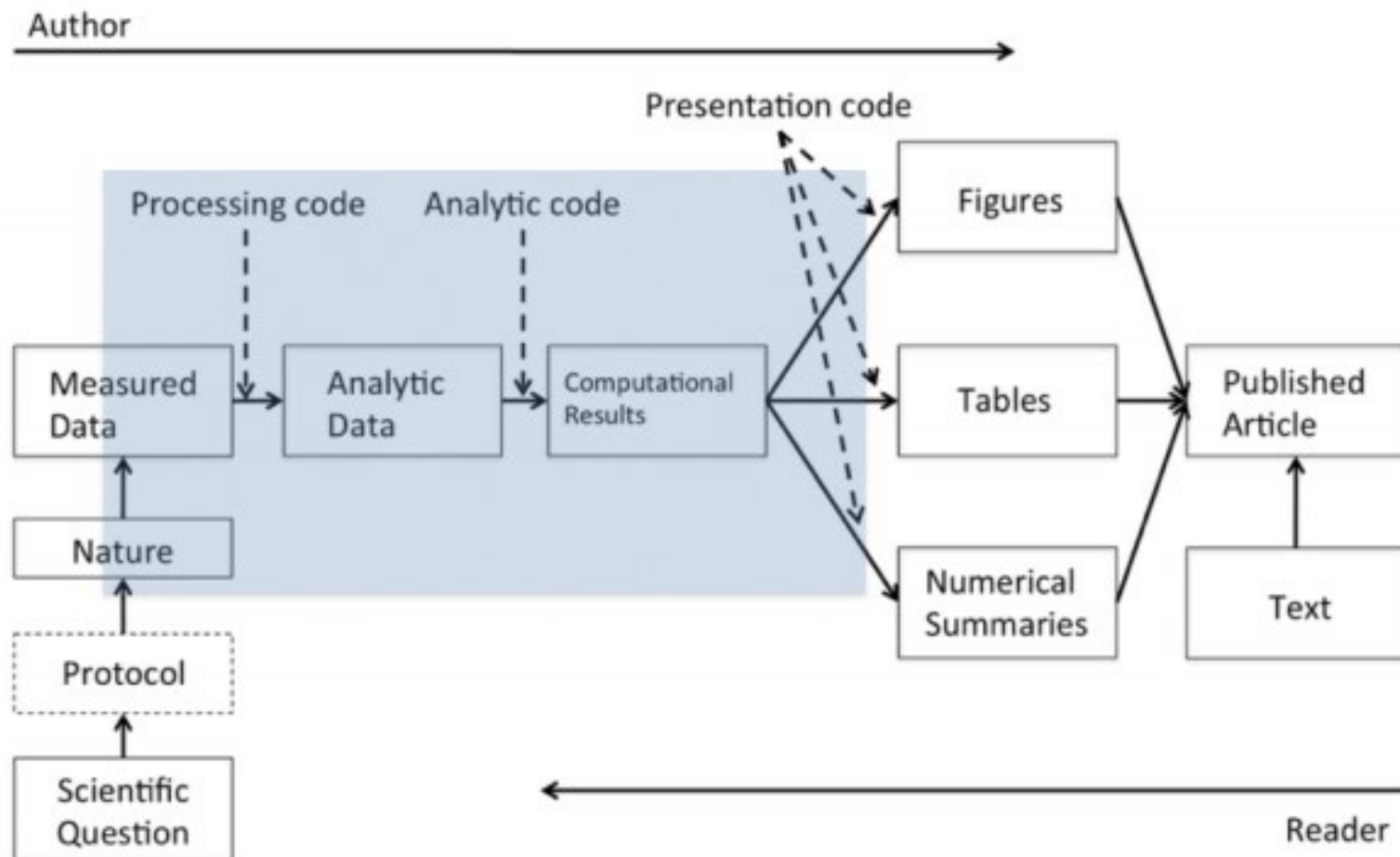


- ▶ Installieren Sie R und RStudio:
https://www.youtube.com/watch?v=_e5PxQVSjXo
- ▶ Probieren Sie – sofern noch nicht gemacht – die in den Folien beschriebenen Befehle. Speichern Sie den Code in einem R-Script.



Berichte

Data Science Pipeline





Berichte

Dynamische Berichte

1. Analysieren der Daten, z.B. mit R
2. Einbau des Codes, der die Resultate (Tabellen, Grafiken, Zahlen) erzeugt in den Bericht
3. Erzeuge Resultate automatisch innerhalb des Codes
4. Wiederholen 1.-3. (2.+3. geschieht automatisch)

Unterstützt in R:

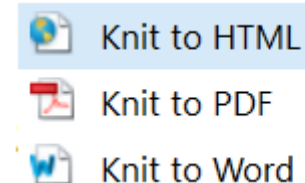
- ▶ Sweave (kombiniert Latex und R-Code)
- ▶ Package `knitr` (Latex/HTML/Markdown und R-Code)
<http://yihui.name/knitr>



Berichte

R Markdown/Quarto

- ▶ Markdown (*.md) ist eine einfache markup-Sprache mit plain text als Formatierungssyntax
- ▶ R Markdown (*.rmd) kombiniert R Code und Markdown-Code und erlaubt das Generieren dynamischer Berichte
- ▶ Quarto-Dateien (*.qmd)
- ▶ R Markdown in RStudio:
 1. Öffnen eines Beispiel-Markdown-Dokuments
`File -> New File -> R Markdown`
 2. Auswählen des Ausgabeformats (HTML, PDF, Word)
 3. Mit `Knit to ...` wird der Bericht erzeugt



R Markdown/Quarto

AUFGABE 2



- ▶ Öffnen eines R Markdown-Dokuments in RStudio
File -> New File -> R Markdown
- ▶ Eingabe der Metainformation und HTML als Output-Format
- ▶ Erzeugen von Reports durch „Knit PDF / HTML / Word“
- ▶ Ändern des YAML-Headers
 - ▶ output: beamer_presentation
 - ▶ output: slidy_presentation
 - ▶ output: ioslides_presentation



R Markdown Struktur

The screenshot shows an R Markdown document in RStudio with the following content and annotations:

```
1 ---
2 title: "Studie A"
3 author: "nomen nomen"
4 date: "24 September 20xx"
5 output: word_document
6 ---
7
8 {r setup, include=FALSE}
9 knitr::opts_chunk$set(echo = TRUE, warning = TRUE)
10
11 ## R Markdown
12
13 This is an R Markdown document. Markdown is a simple formatting syntax for authoring
14 HTML, PDF, and MS Word documents. For more details on using R Markdown see
15 <http://rmarkdown.rstudio.com>.
16 The Document was created with `r version[[13]][1]`.
17
18 {r cars}
19 summary(cars)
```

Annotations:

- YAML Header:** Points to the header block (lines 2-5).
- R-Chunks Ctrl+Alt+I:** Points to the R chunk header (line 8).
- Konfiguration:** Points to the R chunk options (line 9).
- Markdown-Formatierung:** Points to the R Markdown section header (line 11).
- R-Code in Markdown-Text:** Points to the R code embedded in the text (line 15).



R Markdown/Quarto Syntax

`*italics*` and `_italics_`

`**bold**` and `__bold__`

`superscript^2^`

`~~strikethrough~~`

`[link](www.rstudio.com)`

`# Header 1`

`## Header 2`

`### Header 3`

`#### Header 4`

italics and italics

bold and bold

superscript²

~~strikethrough~~

[link](#)

Header 1

Header 2

Header 3

R Markdown/Quarto

Listen



```
* unordered list
* item 2
  + sub-item 1
  + sub-item 2
```

```
1. ordered list
2. item 2
  + sub-item 1
  + sub-item 2
```

- unordered list
- item 2
 - sub-item 1
 - sub-item 2

1. ordered list
2. item 2
 - sub-item 1
 - sub-item 2



R Markdown/Quarto

Sonderelemente

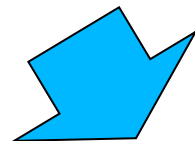
Syntax	Ergebnis
<code>*</code>	*
<code>\epsilon</code>	€
<code>3^{20}</code>	3^{20}
<code>A_b</code>	A_b
<code>\sum_{i=1}^n x_i</code>	$\sum_{i=1}^n x_i$
<code>[Hyperlink] (www.statistik-austria.at)</code>	Hyperlink
<code>![] (path/to/image.png)</code>	



R Markdown/Quarto

Tabellen

```
|Tabellen          |sind              |cool  |
|:-----:         |:-----:         |:-----:|
|Spalte 1 ist      |links-bündig      |$1     |
|Spalte 2 ist      |zentriert          |$12    |
|Spalte 3 ist      |rechts-bündig      |$123   |
```



Tabellen	sind	cool
Spalte 1 ist	links-bündig	\$1
Spalte 2 ist	zentriert	\$12
Spalte 3 ist	rechts-bündig	\$123



R Markdown/Quarto

Einbetten von R-Code

R Code wird in R Chunks eingebettet:

- ▶ R Code zwischen zwei Zeilen mit 3-4 backticks `
- ▶ Am Ende der ersten Zeile {r}
- ▶ Tastaturkürzel: `Strg + Alt + I`

By default

- ▶ wird der Code innerhalb der Chunks im Report ausgegeben
- ▶ wird der Code ausgeführt und der Output nach 2 hashtags `##` angezeigt

Innerhalb des Textes kann R Code jederzeit zwischen `r` und ` angegeben werde. Bsp.: ``r mean(iris$Petal.Length)``



R Markdown/Quarto

R Chunks: Optionen

Optionen innerhalb der {r} Klammern

```
```{r echo=FALSE, eval=FALSE}  
mean(iris$Petal.Length)
```
```

| Option | Ergebnis | Typ |
|-----------|--|--|
| eval | Anzeige der Ausgabe? | Logical (default: TRUE) |
| echo | Anzeige des R-Codes? | Logical (default: TRUE) oder
numerisch (für best. Zeilen) |
| cache | Speichern der R-Objekte
solange Chunk sich nicht ändert | Logical (default: TRUE) |
| fig.align | Ausrichtung von Grafiken | Character, z.B. „center“ |
| out.width | Ändert die Grafikgröße | Character, z.B. „60px“, „7cm“ |
| ... | | |



R Markdown/Quarto

R Chunks: echo & eval

Numerische Werte (oder Vektoren) bei echo oder eval geben an, welche Zeilen angezeigt werden sollen

```
```{r echo=2, eval=FALSE}  
this is a comment
sum(c(1,2,3))
```
```

Damit gibt der Chunk nur die Zeile

```
sum(c(1,2,3))
```

aus (kein Ergebnis, da `eval=FALSE`).

Mit `eval=TRUE` ergibt sich:

```
sum(c(1,2,3))
```

```
## [1] 6
```



R Markdown/Quarto

R Chunks: Einbau von externem R Code

Eine R-Datei `script.R` hat folgenden Inhalt

```
# this is a comment  
sum(c(1,2,3))
```

Mit der Angabe `code=readLines("script.R")` im Chunk

```
````{r echo=TRUE, eval=TRUE,  
code=readLines("Script.R")}
````
```

kommt man zur selben Ausgabe wie auf der letzten Seite:

```
sum(c(1,2,3))  
## [1] 6
```

Unabhängig von Markdown: `source("../Daten//packages.R")`



R Markdown/Quarto

R Chunks: Child Documents

Child documents ...

- ▶ erlauben es Code auszulagern bzw. das Dokument in kleinere Bereiche zu unterteilen
- ▶ erhöhen die Flexibilität (z.B. einfache Änderung der Reihenfolge)

Der Einbau erfolgt allein durch Angabe des child

```
```{r child="Chapter1.Rmd"}  
```
```

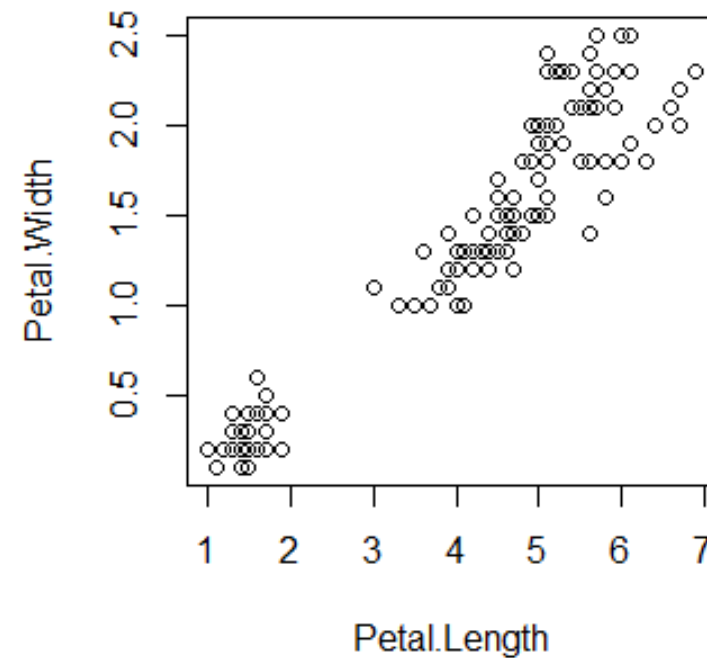
Cave: Keine YAML-Header im Child document



R Markdown/Quarto

R Chunks: Grafiken

```
```{r out.width="120px"}  
plot(iris[,3:4])
```
```





R Markdown/Quarto

R Chunks: Tabellen

```
```{r}
iris[1:2,1:4]
```
```

Erzeugt folgenden Output:

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width
## 1           5.1           3.5           1.4           0.2
## 2           4.9           3.0           1.4           0.2
```

Das knitr Package erlaubt die Erzeugung von Tabellen in Chunks mit der Funktion `kable()`.

```
```{r}
kable(iris[1:2,1:4])
```
```

| | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width |
|--|--------------|-------------|--------------|-------------|
| | 5.1 | 3.5 | 1.4 | 0.2 |
| | 4.9 | 3.0 | 1.4 | 0.2 |

R Markdown/Quarto

AUFGABE 3



- ▶ Einfügen in ein R Markdown Dokument
 - ▶ Satz (nach dem Chunk mit dem `label cars`), welcher die durchschnittliche (`mean`) Geschwindigkeit der Autos angibt. (Hinweis: Verwenden Sie inline R Code)
 - ▶ In einem neuen Chunk (nach dem `label cars`) sollen `dist` und `speed` in einem Streudiagramm (`plot()`) visualisiert werden
 - ▶ Verwenden Sie die Funktion `kable()` um die `summary` der Cars-Daten besser darzustellen
 - ▶ Beschreiben Sie die Daten und die Ergebnisse der Analyse in dieser default-Datei. Formatieren sie den Text (highlighting) mit Markdown
- ▶ Erzeugen Sie eine Datei, in der Sie sich (mit Bild) und Ihr Können kurz präsentieren. Formatieren Sie die Texte in geeigneter Weise und stellen Sie die Informationen übersichtlich, sowohl in Form von Tabellen als auch in Listen, dar.



AUFGABEN

- ▶ Für alle Aufgaben der kommenden Übungszettel müssen folgende Dateien abgegeben werden:
 - ▶ R Script oder Markdown/Quarto-Datei
 - ▶ docx/pdf Datei
- ▶ Wenn angegeben:
 - ▶ Vollständig formatierter Bericht (inkl. Diskussion), wie ihn ein Auftraggeber bekommen würde. Enthält der Bericht auch Daten, immer mit der Funktion `head()`. Dazu gehören u.a. auch
 - ▶ Beschriftung bei Grafiken und Tabellen
 - ▶ Erklärungen gelöschter Daten, etc.
 - ▶ Die gespeicherten Daten (.RData)
- ▶ Die Aufgaben sind vorwiegend mit den Mitteln der bisher besprochenen Kapitel zu lösen.



R als Taschenrechner

```
3+4
```

```
[1] 7
```

```
(3+4) / 2*10
```

```
[1] 35
```

```
a <- 0.5
```

```
b = a*2
```

```
exp(b)
```

```
[1] 2.718
```

```
(cos(log(exp(b))) - a) / b + 1
```

```
[1] 1.04
```



Wichtige Operatoren

+ - * /

Grundrechnungsarten

^

Potenzieren

= <- ->

Zuweisungen

:

Erzeugen von Sequenz ganzer Zahlen

== != < <= ==> logische Vergleiche

#

Kommentarzeichen

```
a <- 1:5
```

```
a
```

```
[1] 1 2 3 4 5
```



Wichtige Operatoren

Die Grundrechnungsarten folgen in der Abarbeitung der math. üblichen Regeln. Zuweisungen und logische Vergleiche haben relativ niedrige Rangordnung. Im Zweifelsfall immer besser ein Klammerpaar mehr als weniger verwenden.

```
2 * 3^4 + 5
```

```
[1] 167
```

```
(2 * (3^4)) + 5
```

```
[1] 167
```

```
2^a < 6
```

```
[1] TRUE  TRUE  FALSE FALSE  FALSE
```

```
100 + 2^a < 6
```

```
[1] FALSE FALSE FALSE FALSE  FALSE
```



Variablen

- ▶ Jede Auswertung kann in einer Variablen gespeichert werden.
- ▶ Es gibt keine vorgegebene Maximalzahl von Variablen
- ▶ Als Variablennamen können beliebige Kombinationen aus Buchstaben, Ziffern, Punkt und Unterstrich verwendet werden
 - ▶ Variablennamen dürfen nicht mit einer Ziffer beginnen.
- ▶ R ist Case-Sensitiv
- ▶ Keine Deklaration notwendig



Datentypen in R

Numerische Vektoren

Üblicherweise wird **c()** zum Erstellen von mehrwertigen Vektoren verwendet. Weitere wichtige Funktionen sind **seq()** zum Erstellen von Sequenzen und **rep()** für Wiederholungen.

```
v1 <- c(1, 3.14, 17)
```

```
v1
```

```
[1] 1.00 3.14 17.00
```

```
v2 <- seq(from=-pi, to=pi, length=5); v2
```

```
[1] -3.141593 -1.570796 0.000000 1.570796 3.141593
```

```
v3 <- rep(2, 5)
```

```
v3
```

```
[1] 2 2 2 2 2
```

```
length(v3)
```

```
[1] 5
```

```
mode(v3)
```

```
[1] "numeric"
```



Datentypen in R

AUFGABE 4: Vektoren erzeugen

Weiters kann ein Vektor ohne Inhalt mit dem Namen des atomaren Datentyps erzeugt werden.

- ▶ Erzeugen Sie für jeden atomaren Datentyp einen Vektor mit 10 Einträgen
- ▶ Erzeugen Sie einen Vektor, der die ersten 10 Quadratzahlen enthält
- ▶ Gegeben ist folgender Vektor

```
temperaturen <- c(15, 18, 22, 20, 17, 25, 28, 19, 14, 21)
```

- ▶ Filtern Sie mit `subset()` alle Temperaturen unter 20°C
- ▶ Konvertieren Sie diese Werte in Fahrenheit = $(\text{Celsius} * 9/5) + 32$
- ▶ Berechnen Sie den Durchschnitt der °F mit `mean()`
- ▶ Geben Sie alle Vektoren aus
- ▶ Lösung: MW = 61.88



Datentypen in R

Faktoren

Nominale und ordinale Daten sind in R „Faktoren“. Intern ist das ein ganzzahliger Vektor, bei dem jeder Zahl ein „Level“ zugeordnet ist.

```
behandlung <- rep(c("Placebo", "Verum"), c(2,3))
```

```
behandlung
```

```
[1] "Placebo" "Placebo" "Verum"    "Verum"    "Verum"
```

```
summary(behandlung)
```

| Length | Class | Mode |
|--------|-----------|-----------|
| 5 | character | character |

```
behandlung <- factor(behandlung)
```

```
behandlung
```

```
[1] Placebo    Placebo Verum      Verum      Verum
```

```
Levels: Placebo Verum
```



Datentypen in R

Faktoren Vorteile

- ▶ Effizienter Speicherverbrauch
- ▶ Einfache Datenanalyse: automatische Gruppierung
- ▶ Konsistenz der Daten-Kompatibilität mit Modellen
- ▶ Bessere Lesbarkeit
- ▶ Unterstützung ordinaler Daten
 - ▶ Darstellung $a < b < c$
 - ▶ Bessere statistische Analysen
 - ▶ Automatische Sortierung z.B. Visualisierungen
 - ▶ Logische Vergleiche



Datentypen in R

Faktoren

```
summary(behandlung)
  Placebo      Verum
        2         3
```

```
unique(behandlung)
[1] Placebo Verum
Levels: Placebo Verum
```

```
levels(behandlung)
[1] "Placebo" "Verum"
```

Behält Faktor bei

Faktorlevel als Character



Datentypen in R

Geordnete Faktoren

Ordinale Messgrößen werden durch geordnete Faktoren dargestellt:

```
zustand <- ordered(c("Gut", "Schlecht", "Super", "Super", "Frag nicht", "Gut"),  
                  levels = c("Frag nicht", "Schlecht", "Gut", "Super"))
```

Zustand

```
[1] Gut      Schlecht Super    Super    Frag nicht Gut
```

```
Levels: Frag nicht < Schlecht < Gut < Super
```

Es ist wichtig, nominale und ordinale Variablen als solche zu kodieren, da viele Statistik-Funktionen diese von Zeichenvektoren unterscheiden. Beim Datenimport wird dies aber ohnehin fast immer automatisch erledigt.



Datentypen in R (Geordnete) Faktoren mit cut()

Eine sehr nützliche Funktion ist cut():
Numerischer Vektor wird nach definierten Grenzen unterteilt

```
alter <- c(25, 30, 42, 18, 55, 60, 22, 37, 28, 48)
grenzen <- c(0, 20, 30, 40, 50, 100)
alter_faktor <- cut(alter,
                    breaks = grenzen,
                    labels = c("0-20", "21-30", "31-40", "41-50", "51+"))
alter_faktor

[1] 21-30 21-30 41-50 0-20 51+ 51+ 21-30 31-40 21-30
[10] 41-50
Levels: 0-20 21-30 31-40 41-50 51+
```

Probieren Sie den Parameter `ordered=TRUE`



Datentypen in R

AUFGABE 5: Vektoren & Faktoren

Probieren Sie – sofern noch nicht gemacht – die in den Folien beschriebenen Befehle.



Datentypen in R

Datenmatrix

Die wohl wichtigste Struktur zur Haltung von Daten im üblichen Rechteckschema mit Zeilen und Spalten ist die Datenmatrix. In R wird diese `data.frame` genannt.

```
data(iris)
```

```
class(iris)
```

```
[1] "data.frame"
```

```
head(iris)
```

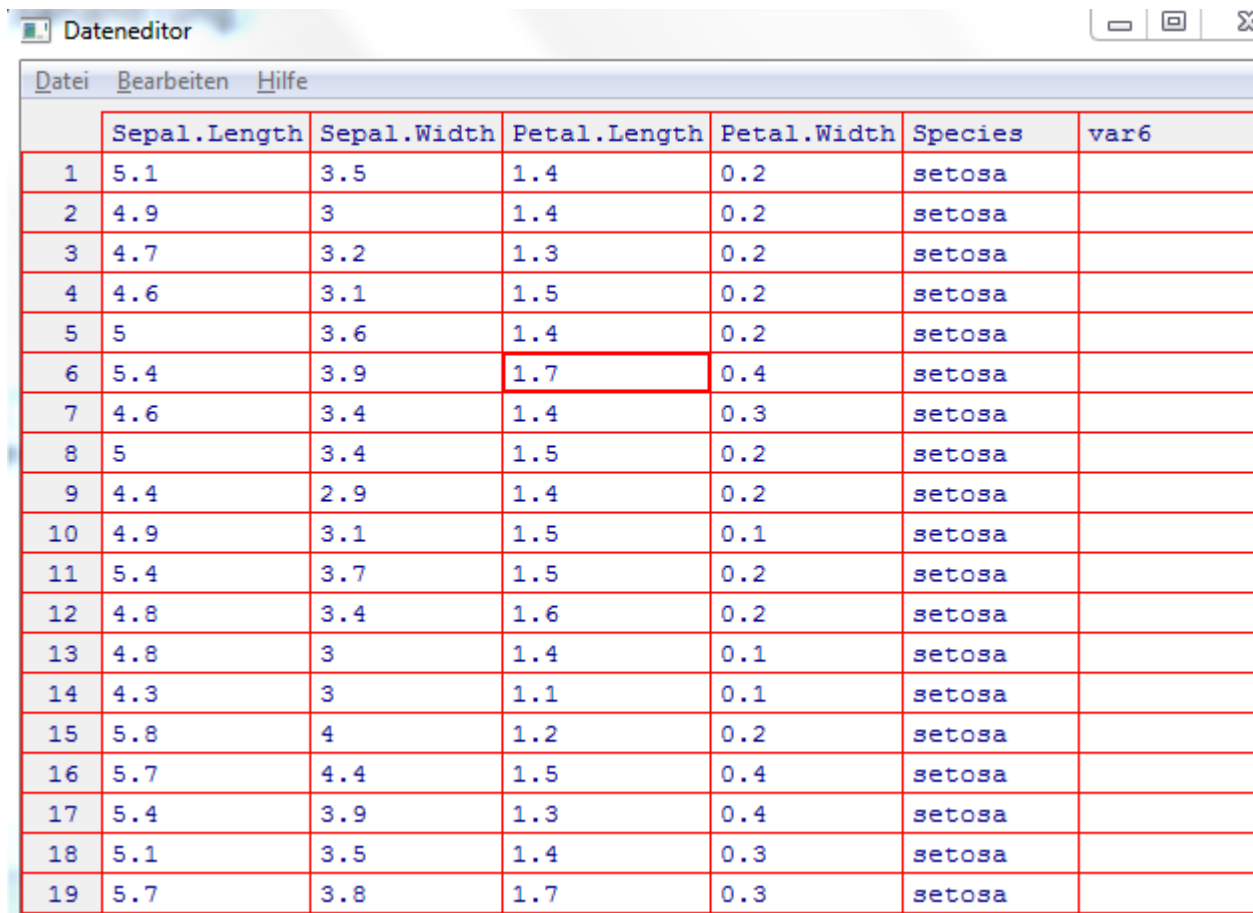
| | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 2 | 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 3 | 4.7 | 3.2 | 1.3 | 0.2 | setosa |
| 4 | 4.6 | 3.1 | 1.5 | 0.2 | setosa |
| 5 | 5.0 | 3.6 | 1.4 | 0.2 | setosa |
| 6 | 5.4 | 3.9 | 1.7 | 0.4 | setosa |



Datentypen in R

Datenmatrix

```
edit(iris)
```



The screenshot shows the 'Dateneditor' window in R, which is a data editor for the 'iris' dataset. The window has a menu bar with 'Datei', 'Bearbeiten', and 'Hilfe'. Below the menu bar is a table with 7 columns: 'Sepal.Length', 'Sepal.Width', 'Petal.Length', 'Petal.Width', 'Species', and 'var6'. The table contains 19 rows of data. The cell containing '1.7' in the 'Petal.Length' column of the 6th row is highlighted with a red border. The 'Species' column contains the value 'setosa' for all rows.

| | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species | var6 |
|----|--------------|-------------|--------------|-------------|---------|------|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | setosa | |
| 2 | 4.9 | 3 | 1.4 | 0.2 | setosa | |
| 3 | 4.7 | 3.2 | 1.3 | 0.2 | setosa | |
| 4 | 4.6 | 3.1 | 1.5 | 0.2 | setosa | |
| 5 | 5 | 3.6 | 1.4 | 0.2 | setosa | |
| 6 | 5.4 | 3.9 | 1.7 | 0.4 | setosa | |
| 7 | 4.6 | 3.4 | 1.4 | 0.3 | setosa | |
| 8 | 5 | 3.4 | 1.5 | 0.2 | setosa | |
| 9 | 4.4 | 2.9 | 1.4 | 0.2 | setosa | |
| 10 | 4.9 | 3.1 | 1.5 | 0.1 | setosa | |
| 11 | 5.4 | 3.7 | 1.5 | 0.2 | setosa | |
| 12 | 4.8 | 3.4 | 1.6 | 0.2 | setosa | |
| 13 | 4.8 | 3 | 1.4 | 0.1 | setosa | |
| 14 | 4.3 | 3 | 1.1 | 0.1 | setosa | |
| 15 | 5.8 | 4 | 1.2 | 0.2 | setosa | |
| 16 | 5.7 | 4.4 | 1.5 | 0.4 | setosa | |
| 17 | 5.4 | 3.9 | 1.3 | 0.4 | setosa | |
| 18 | 5.1 | 3.5 | 1.4 | 0.3 | setosa | |
| 19 | 5.7 | 3.8 | 1.7 | 0.3 | setosa | |



Datentypen in R

Datenmatrix

Die Spalten von data frames beinhalten in der Regel numerische Vektoren oder Faktoren. Mit `str()` und `summary()` kann man feststellen, ob nominale oder ordinale Messgrößen tatsächlich als solche verwendet werden.

```
str(iris, vec.len=1)
```

```
'data.frame': 150 obs. of 5 variables:
```

```
 $ Sepal.Length: num  5.1 4.9 ...
```

```
$ Sepal.Width : num  3.5 3 ...
```

```
$ Petal.Length: num  1.4 1.4 ...
```

```
$ Petal.Width : num  0.2 0.2 ...
```

```
$ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 ...
```



Datentypen in R

Datenmatrix

Die Spalten von data frames beinhalten in der Regel numerische Vektoren oder Faktoren. Mit `str()` und `summary()` kann man feststellen, ob nominale oder ordinale Messgrößen tatsächlich als solche verwendet werden.

```
summary(iris)
```

| Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---------------|---------------|---------------|---------------|---------------|
| Min. :4.300 | Min. :2.000 | Min. :1.000 | Min. :0.100 | setosa :50 |
| 1st Qu.:5.100 | 1st Qu.:2.800 | 1st Qu.:1.600 | 1st Qu.:0.300 | versicolor:50 |
| Median :5.800 | Median :3.000 | Median :4.350 | Median :1.300 | virginica :50 |
| Mean :5.843 | Mean :3.057 | Mean :3.758 | Mean :1.199 | |
| 3rd Qu.:6.400 | 3rd Qu.:3.300 | 3rd Qu.:5.100 | 3rd Qu.:1.800 | |
| Max. :7.900 | Max. :4.400 | Max. :6.900 | Max. :2.500 | |



Datentypen in R

Datenmatrix

Erstellen eines data.frames über

```
data <- data.frame(var1=v3, var2=v2)
```

data

| | var1 | var2 |
|---|----------|-----------|
| 1 | 2 | -3.141593 |
| 2 | 2 | -1.570796 |
| 3 | 2 | 0.000000 |
| 4 | 2 | 1.570796 |
| 5 | 3.141593 | |

Einlesen aus csv

```
df <- read.csv („Datei.csv“)
```



Datentypen in R

Teilmengen Datenmatrix

Mit eckigen Klammers können Zeilen und Spalten aus data frames extrahiert werden. Die beiden Indexausdrücke werden durch ein Komma getrennt. Es sind dieselben Indizierungen wie für einfache Vektoren möglich.

```
data(iris)
```

```
dim(iris)
```

```
[1] 150  5
```

```
summary(iris[1:50, 1:2])
```

| Sepal.Length | Sepal.Width |
|---------------|---------------|
| Min. :4.300 | Min. :2.300 |
| 1st Qu.:4.800 | 1st Qu.:3.200 |
| Median :5.000 | Median :3.400 |
| Mean :5.006 | Mean :3.428 |
| 3rd Qu.:5.200 | 3rd Qu.:3.675 |
| Max. :5.800 | Max. :4.400 |



Datentypen in R

Teilmengen Datenmatrix

Auf benannte Spalten eines data frames kann auch einfacher mit dem `$`-Operator zugegriffen werden.

```
iris$Sepal.Width
```

```
[1] 3.5 3.0 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 3.7 3.4 3.0 3.0 4.0 4.4 3.9 3.5 3.8 3.8 3.4  
[22] 3.7 3.6 3.3 3.4 3.0 3.4 3.5 3.4 3.2 3.1 3.4 4.1 4.2 3.1 3.2 3.5 3.6 3.0 3.4 3.5 2.3  
[43] 3.2 3.5 3.8 3.0 3.8 3.2 3.7 3.3 3.2 3.2 3.1 2.3 2.8 2.8 3.3 2.4 2.9 2.7 2.0 3.0 2.2  
[64] 2.9 2.9 3.1 3.0 2.7 2.2 2.5 3.2 2.8 2.5 2.8 2.9 3.0 2.8 3.0 2.9 2.6 2.4 2.4 2.7 2.7  
[85] 3.0 3.4 3.1 2.3 3.0 2.5 2.6 3.0 2.6 2.3 2.7 3.0 2.9 2.9 2.5 2.8 3.3 2.7 3.0 2.9 3.0  
[106] 3.0 2.5 2.9 2.5 3.6 3.2 2.7 3.0 2.5 2.8 3.2 3.0 3.8 2.6 2.2 3.2 2.8 2.8 2.7 3.3 3.2  
[127] 2.8 3.0 2.8 3.0 2.8 3.8 2.8 2.8 2.6 3.0 3.4 3.1 3.0 3.1 3.1 3.1 2.7 3.2 3.3 3.0 2.5  
[148] 3.0 3.4 3.0
```



Datentypen in R

Teilmengen Datenmatrix

Auswahl eines Teils einer Spalte

```
iris[iris$Species == "setosa", "Sepal.Width"]  
[1] 3.5 3.0 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 3.7 3.4 3.0 3.0  
[15] 4.0 4.4 3.9 3.5 3.8 3.8 3.4 3.7 3.6 3.3 3.4 3.0 3.4 3.5  
[29] 3.4 3.2 3.1 3.4 4.1 4.2 3.1 3.2 3.5 3.6 3.0 3.4 3.5 2.3  
[43] 3.2 3.5 3.8 3.0 3.8 3.2 3.7 3.3
```

```
iris[iris$Species == "setosa",]  
iris[iris$Sepal.Width < 2.4,]  
any(iris$Sepal.Width<2.4)  
any(iris$Sepal.Width<2.4)
```



Datentypen in R

AUFGABE 6: Data Frame

Probieren Sie – sofern noch nicht gemacht – die in den Folien beschriebenen Befehle.



Datentypen in R

AUFGABE 7: Data Frame erzeugen

- ▶ Legen Sie ein d.f. mit x, Sex, Age, weight, height und 6 Einträgen (zufällig oder manuell) an
- ▶ Fügen Sie eine Spalte „Name“ hinzu
- ▶ Wo sinnvoll wandeln Sie die Spalten in einen Faktor um
- ▶ Ändern Sie den Namen „x“ auf „ID“
- ▶ Generieren Sie verschiedene Teilausgaben, z.B.
 - ▶ Zeile 2 und 3
 - ▶ Zeile 2 und 3, Spalte 4 bis 6
 - ▶ Nur die Spalte „weight“
 - ▶ Alle Personen mit Alter < 30



Data.Frame subset

- ▶ Ein gleiches Ergebnis liefert auch die Funktion `subset()`

```
subset(iris, Species=="setosa")
```

- ▶ `subset()` kann ebenfalls erweitert werden

```
subset(iris, Species=="setosa", select=-5)
```

```
subset(iris, Species %in% c("setosa", "virginica"))
```

```
subset(iris, Species %in% c("setosa", "virginica") &  
Sepal.Width<2.4)
```

- ▶ Sehr wichtig ist das Auftrennen eines Data.Frames in Subgruppen

```
LSpecies <- split(iris, iris$Species)
```



apply-Funktionen

apply()

- ▶ Die apply-Familie ist besonders mächtig für die Anwendung von Funktionen auf Datenstrukturen

```
apply(X = cars, MARGIN = 2, FUN = max)
apply(cars, 2, quantile, 0.25)
span <- function(x){
  return(max(x)-min(x))
}
apply(cars, 2, span)
```

- ▶ **Anonyme Funktion**

```
apply(cars, 2, function(x) {span(x)})
apply(cars, 2, function(x) {max(x)-min(x)})
```



apply-Funktionen

apply-Familie

| Funktion | | Input | Output |
|--|---|---------------------------------|---------------------------|
| <code>apply(X, MARGIN, FUN)</code> | Wendet Funktion auf Spalte oder Zeile an | matrix
array
data.frame | vector,
array,
list |
| <code>lapply(X, FUN)</code> | Ähnlich <code>apply</code> aber immer Einträge/Spalten.
Verlässlicher Output | vector, m
list
data.frame | list |
| <code>sapply(X, FUN)</code> | Ähnlich <code>lapply</code> ,
versucht Ergebnis zu vereinfachen
gut geeignet wenn Ergebnis skalar ist | vector, m
list
data.frame | vector
array
list |
| <code>vapply(X, FUN, FUN.V)</code> | Ähnlich <code>sapply</code> ,
explizite Festlegung Ergebnistyp mit <code>FUN.VALUE</code> | list
data.frame | vector
array
list |
| Weitere: <code>tapply</code> , <code>mapply</code> , <code>rapply</code> | | | |



apply-Funktionen

AUFGABE 8: `sapply()`



- ▶ Laden Sie den Datensatz „airquality“ aus dem Package „datasets“
- ▶ Bestimmen Sie die Mittelwerte für „Ozone“ und „Wind“ in den einzelnen Monaten mit einem apply-Befehl.



apply-Funktionen

aggregate()

- ▶ `aggregate()` $\approx$ `tapply()`
- ▶ Hauptanwendung für `data.frames`
- ▶ Formulanotation möglich
- ▶ auch >1 Variablen
- ▶ auch >1 Funktion (auch möglich mit `tapply`)

```
aggregate(x=iris$Sepal.Length, by=list(iris$Species), mean)
aggregate(Sepal.Length ~ Species, iris, mean)
aggregate(iris[,c("Sepal.Length", "Sepal.Width")],
  list(iris$Species), mean)
aggregate(cbind(Sepal.Width, Sepal.Length) ~ Species, iris, mean)
aggregate(Sepal.Length ~ Species, iris, FUN=function(x)
{c(MW=mean(x), MEDIAN=median(x))})
```



apply-Funktionen

AUFGABE 9: aggregate()

- ▶ Berechnen Sie im Diamant-Datensatz (ggplot2)
 - ▶ den Mittelwert der abhängigen Variablen „price“,
 - ▶ sowie den Median, kategorisiert nach „cut“
 - ▶ sowie den Median, kategorisiert nach „cut“ und „color“
 - ▶ und der abhängigen Variablen „carat“, kategorisiert nach
 - ▶ „cut“
 - ▶ „cut“ und „color“



dplyr

Allgemeines

Strg + Shift + m

► Kohärente Syntax `%>%` oder `|>`

► Lesbarkeit

```
dim(head(iris, n=4))
```

```
iris %>% head(4) %>% dim
```

► Effizienz (C++)

► Paketkonsistenz: Teil von Tidyverse

► Keine neue Funktionalität





dplyr Vergleich

| Kriterium | data.frame | tibble | data.table |
|-------------|-------------------------------------|-------------------------------------|-------------------------------------|
| Performance | ☐ | ☐ | ☒ |
| Sicherheit | ☒ | ☒ | ☐ |
| tidyverse | ☐ | ☒ | ☒ |
| Speicher | ☐ | ☒ | ☒ |
| Syntax | klassisch | modern | speziell |



dplyr

Einfache Funktionen

- ▶ **Distinct:** Rückgabe nicht-redundanter Zeilen

```
iris %>% distinct %>% nrow
```

- ▶ **Slice:** Selektiere Zeilen nach Indices

```
tbl.iris %>% slice(c(1:3, 2, 5:10))
```

- ▶ **Sample:** Zufällige Auswahl an Zeilen

```
iris %>% sample_frac(0.1, replace=F)
```

```
iris %>% sample_n(20, replace=F)
```



dplyr **select**

► Spaltenauswahl von data.frame, data.table, tibble

```
select(data=iris, Species, Sepal.Width)
```

```
iris %>% select(Species, Sepal.Width)
```

```
iris %>% select(Species:Sepal.Width)
```

```
iris %>% select(!Species:Sepal.Width)
```

```
iris %>% select(-Species, -Sepal.Width)
```

► Speichern von Spalten in einem Vektor

```
usedCols <- c("Species", "Sepal.Width")
```

```
iris %>% select(all_of(usedCols))
```

```
iris %>% select(any_of(usedCols))
```



dplyr **filter**

- ▶ Auswahl von Zeilen auf Grund logischer Expression
- ▶ Schneller (nur) bei großen Datensätzen

```
iris %>% filter(Species=="setosa")
```

```
iris %>% filter(Sepal.Width > mean(Sepal.Width))
```

```
iris %>% filter(between(Sepal.Width, 1.5, 2.5))
```

```
iris %>% filter(Sepal.Width>4 & Sepal.Length>4)
```



dplyr **summarize**

- ▶ Zum Aggregieren von Daten
- ▶ z.B. Berechnen von Mittelwerten

```
iris %>% summarize(AvgSW = mean(Sepal.Width),  
                  MedianSW = median(Sepal.Width))
```

```
iris %>% dplyr::summarise(across(1:4, mean))
```

```
iris %>% summarize(across(-Species, mean))
```

```
iris %>% summarize(across(where(is.numeric), mean))
```

```
iris %>% count("Species")
```

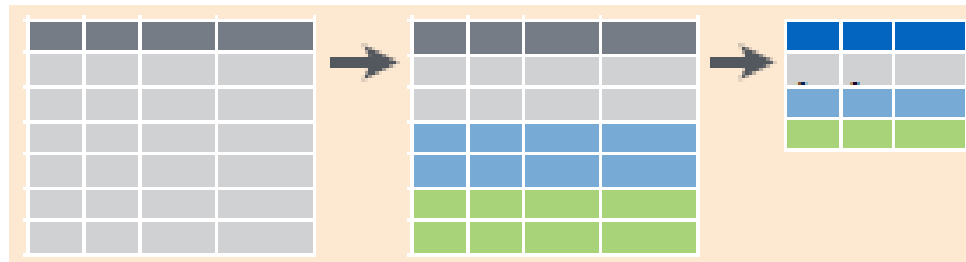
```
iris %>% count(Species, Sepal.Width)
```



dplyr group_by

- ▶ Gruppiert Daten nach Variablen
- ▶ Meist mit summarize()-Funktion

```
iris %>% group_by(Species) %>%  
  summarize(AvgSW = mean(Sepal.Width),  
            medianMV = median(Sepal.Width)) %>%  
  arrange(desc(AvgSW))
```





Datentypen in R

AUFGABE 10: dplyr

Probieren Sie – sofern noch nicht gemacht – die in den Folien beschriebenen Befehle.



dplyr

AUFGABE 11

- ▶ Wählen Sie die Spalten cut, color, carat und price aus diamonds
- ▶ Filtern Sie nach Diamanten mit „Premium“ cut und carat > 1.0
- ▶ Gruppieren Sie die Daten nach der color-Spalte
- ▶ Berechnen Sie den durchschnittlichen Preis für jede color-Gruppe
- ▶ Das Ergebnis soll nach absteigendem Preis ausgegeben werden.



dplyr **mutate**

- ▶ Erzeugt eine neue (berechnete) Spalte

```
iris %>% mutate(Sepal.Width/Sepal.Length)
iris %>% select(Sepal.Width, Sepal.Length) %>%
  mutate(Ratio = Sepal.Width/Sepal.Length)
```

CAVE: Daten werden nicht gespeichert



dplyr

Datensatz kombinieren

```
iris1 <- iris %>% mutate(id=c(1:nrow(iris))) %>% head(n=5)
iris2 <- dplyr::select(iris, -Species) %>% head(n=5)
iris2$id <- c(1:2, 6:8)
```

| a | | b | |
|----|----|----|----|
| x1 | x2 | x1 | x3 |
| A | 1 | A | T |
| B | 2 | B | F |
| C | 3 | D | T |

+

=

```
(x <- left_join(iris1, iris2, by="id"))
(x <- right_join(iris1, iris2, by="id"))
(x <- inner_join(iris1, iris2, by="id"))
(x <- full_join(iris1, iris2, by="id"))
```

| x1 | x2 | x3 |
|----|----|----|
| A | 1 | T |
| B | 2 | F |
| C | 3 | NA |

| x1 | x3 | x2 |
|----|----|----|
| A | T | 1 |
| B | F | 2 |
| D | T | NA |

| x1 | x2 | x3 |
|----|----|----|
| A | 1 | T |
| B | 2 | F |

| x1 | x2 | x3 |
|----|----|----|
| A | 1 | T |
| B | 2 | F |
| C | 3 | NA |
| D | NA | T |

tidyr

Umstrukturieren



► Bereinigung und Umstrukturierung der Daten

► Gather

► Breite -> Lange Form

► Spalten -> Zeilen

```
beat <- data.frame(
  name = c("Hans", "Julia", "Anton"),
  control = c(68, 80, 66),
  treatment = c(57, 87, 50)
)
long_beat <- beat %>% pivot_longer(
  cols=c("control", "treatment"),
  names_to = "drug",
  values_to = "heartrate"
)
```

| name | control | treatment |
|-------|---------|-----------|
| Hans | 68 | 57 |
| Julia | 80 | 87 |
| Anton | 66 | 50 |



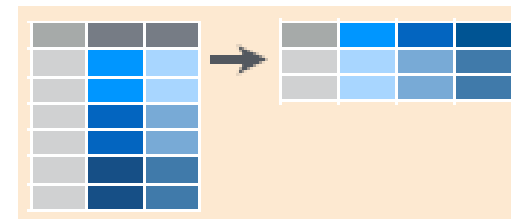
| name | drug | heartrate |
|-------|-----------|-----------|
| Hans | control | 68 |
| Julia | control | 80 |
| Anton | control | 66 |
| Hans | treatment | 57 |
| Julia | treatment | 87 |
| Anton | treatment | 50 |



tidyr

Umstrukturieren

- ▶ Spread
- ▶ Lange -> Breite Form
- ▶ Zeilen -> Spalten



```
wide_beat <- long_beat %>% pivot_wider(  
  names_from = c("drug"),  
  values_from = c("heartrate")  
)
```

| name | drug | heartrate |
|-------|-----------|-----------|
| Hans | control | 68 |
| Julia | control | 80 |
| Anton | control | 66 |
| Hans | treatment | 57 |
| Julia | treatment | 87 |
| Anton | treatment | 50 |



| name | control | treatment |
|-------|---------|-----------|
| Hans | 68 | 57 |
| Julia | 80 | 87 |
| Anton | 66 | 50 |



Datentypen in R

AUFGABE 12: dplyr & tidyr

Probieren Sie – sofern noch nicht gemacht – die in den Folien beschriebenen Befehle.



tidyr

AUFGABE 13

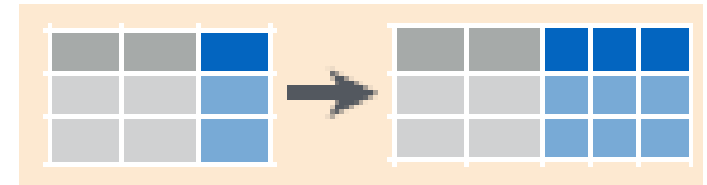
- ▶ Der Iris-Datensatz liegt im Breit-Format vor.
 - ▶ Wandeln Sie diesen in das Lang-Format um,
 - ▶ anschließend wieder zurück in das Breit-Format.
- ▶ Werten Sie den Datensatz „strategie.csv“ induktiv statistisch aus (optimal im long- und wide-Format)



tidyr

Teile Datensatz

- ▶ Separate
 - ▶ Teilt eine Spalte in mehrere auf



```
sep_iris <- long_iris %>%  
  separate(Attribute,  
    into=c("Blatt", "Dim"), sep="\\.")
```

```
dim(sep_iris)
```

```
head(sep_iris)
```

```
sep_iris[c(1:3, 299:302, 598:600),]
```

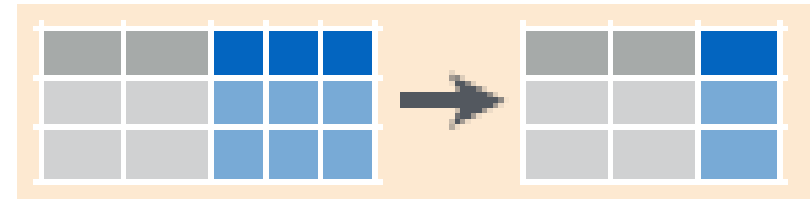


tidyr

Vereinige Datensatz

► Unite

- Vereint mehrere Spalten zu einer



```
unite_iris <- unite(sep_iris, col="Leaf",  
  c("Blatt", "Dim"), sep=".")
```



tidyr

AUFGABE 14

- ▶ Gegeben ist eine Datensatz im Wide-Format

```
measure <- data.frame(  
  id = 1:4,  
  trt = sample(rep(c('control', 'treatment'))),  
  work.T1 = runif(4),  
  home.T1 = runif(4),  
  work.T2 = runif(4),  
  home.T2 = runif(4)  
)
```

- ▶ Bringen Sie den Datensatz in das Long-Format (und vv?)
- ▶ Speichern Sie die location (work|home) sowie die Zeitpunkte (T1|T2) in getrennten Spalten (und vv)